

Reproducing and Extending Price-Bin Prediction for Funko Pop Collectibles

PES2UG24CS655 - Rosalin Verma

PES2UG24CS638 - Keerthana M

Sem 5K

1. Problem Statement

Limited-edition Funko Pop figures sell at a fixed primary-market price but resell at widely varying prices, and sellers have little transaction history to price them. The reference paper (Lu, Application of Machine Learning Methods to Evaluate Collectibles) frames this as 5-class price-bin classification rather than regression, because prices are heavily right-skewed. Our task had two parts:

- Reproduce the paper's linear SVM, custom-kernel SVM, decision tree and bagged decision tree.
- Add one model the paper does not use, and compare it under identical conditions.

2. Dataset Details

- Source: StockX secondary-market sales, 359 items after dropping those with no transactions.
- Features: category (5 values), material (4), special_version (49) and search_volume, a proxy for public interest.
- Target: average sale price split into 5 equal-width log-scale bins (edges $\approx$ \$14, 38, 105, 287, 786, 2150). Bin sizes are 146 / 153 / 46 / 10 / 4, matching the paper, so the imbalance is severe.
- Preprocessing: search volume is log-transformed and min-max scaled. One-hot encoding (59 features) feeds the SVMs, trees and XGBoost; ordinal encoding (4 features) feeds the custom kernel.
- Evaluation: stratified 4-fold CV (shuffle, random_state=1) with folds saved once and shared by every model. Metrics are weighted precision, recall and F1 plus per-class precision, averaged over folds.

3. Approach

Part 1: reproduction. We implemented the paper's pipeline as described:

- Linear SVM (one-vs-one), plus an RBF variant to check that RBF does not help.
- Custom-kernel SVM: Gaussian similarity for search volume; for categorical features, $h(P(z)) = (1 - P^\alpha)^{1/\alpha}$ when two values match and 0 otherwise; composite kernel is a weighted sum with weights [2, 1, 1, 3]; category probabilities come from training folds only. The paper does not state α , θ or C , so we used $\alpha = 0.5$ and $\theta = 1.0$ with the default C .
- Decision tree (random_state=1) and bagged decision tree with 2 to 200 trees.

Part 2: XGBoost (extension). The paper's single tree overfits badly (train F1 0.98 vs. valid 0.41). Boosting is a regularised alternative to bagging, it accepts per-sample weights for the rare high-price bins, and it handles one-hot plus numeric features without scaling. The headline model uses library defaults (multi:softprob, random_state=1). We also tried class-balanced weights, a shallow version (depth 3, lr 0.1, 100 trees), and both together.

4. Implementation Overview

Python with scikit-learn, XGBoost, pandas, NumPy and matplotlib. Notebook 01 does binning, encoding and folds; 02–05 are the reference models; 06 is XGBoost; 07 holds the comparison tables. Every model runs through the shared src/evaluate.py on the same saved folds, and results are written to results/*.csv.

5. Results

Reproduction (validation, weighted P / R / F1):

Model	Reported	Ours
Linear SVM	0.408 / 0.482 / 0.440	0.408 / 0.482 / 0.440
Custom-kernel SVM	0.390 / 0.460 / 0.421	0.392 / 0.462 / 0.423
Decision tree	0.408 / 0.415 / 0.409	0.408 / 0.415 / 0.409
Bagged DT (100 trees)	0.452 / 0.476 / 0.456	0.452 / 0.476 / 0.456

The paper shows only a plot for bagging; this reference comes from the paper author's notebook.

- The linear SVM, decision tree and bagged DT reproduce exactly.
- Custom kernel: validation is within 0.002 of the paper; train F1 is 0.574 vs. 0.602 reported, likely because of the unstated α , θ and C .
- The decision tree matched only with `random_state=1`; an unseeded run gave F1 0.399 because tied splits are broken randomly.
- RBF validation F1 is 0.438 vs. 0.440 for linear, consistent with the paper's claim that RBF does not help.

Extension (validation):

Model	Train F1	Valid P / R / F1	Bin 3 / 4 / 5 precision
Bagged DT (100)	0.983	0.452 / 0.476 / 0.456	0.301 / 0 / 0
XGBoost (defaults)	0.962	0.452 / 0.459 / 0.452	0.329 / 0.375 / 0
XGBoost shallow	0.709	0.448 / 0.482 / 0.454	0.308 / 0 / 0
Linear SVM	0.576	0.408 / 0.482 / 0.440	0 / 0 / 0
Decision tree	0.983	0.408 / 0.415 / 0.409	0.136 / 0 / 0.333

6. Conclusions

- Reproduction succeeded. Three of the four models match to three decimals, and the custom kernel is within 0.002 on validation. Its train gap likely comes from the unstated α , θ and C .
- XGBoost vs. the reference models. It beats the custom-kernel SVM (+0.029 F1) and the single decision tree (+0.043), and is similar to the linear and RBF SVMs and to bagged DT (differences under 0.02, within noise). It is the only model with non-zero Bin 4 precision (0.375), but Bin 4 has only 10 items, so this is suggestive, not conclusive.
- Why it does not clearly beat bagging. Default XGBoost still overfits (train 0.96 vs. valid 0.45); the shallow variant cuts the train/valid gap from 0.51 to 0.26 at the same validation F1, and class-balanced weights did not raise F1.
- Main limitation. No model learns Bin 5 (4 items), and all models score between 0.41 and 0.46 weighted F1. The ceiling comes from the data (359 items, four weak features, as the paper also concludes), not from the choice of algorithm.
- Future work. Richer features (fan-forum text, release date), more data, and imbalance-specific methods such as SMOTE or cost-sensitive learning.